ONNX

Special Interest Group Updates:

Architecture & Infrastructure

Year in Review

ONNX 1.18 to 1.22 · May 2025 – June 2026

Dr. Andreas Fehlner

ONNX Annual Community Meetup — 26th June 2026

Organisational change — Predictable Quarterly Cadence

SIG chairs: Andreas Fehlner (since January 2025), Christian Bourjau (since January 2026)

Release cadence & velocity

– Established better plannable time-based release branch cut-off (every quarter)
– Earlier appointment of release managers (incl. better onboarding)
– Introduction of deprecation rules for dependencies
– Planned to announce major changes 2 releases in advance
– onnx-weekly on PyPI for continuous feedback between stable releases

RFC process for large proposals

– RFC template introduced: structured community review before implementation begins

Releasing Smarter: Our Transition to a Predictable Quarterly Cadence

Release	Date	Release Manager	PyPI Wheels
1.18.0 Q2/25	8 May 2025	Ramakrishnan Sivakumar (AMD)	cp39–cp313 · cp313t (mac/win only)
1.19.0 Q3/25	26 August 2025	Yuan Yao (NVIDIA)	+ cp313t all platforms · + cp314 (linux arm)
1.20.0 Q4/25	1 December 2025	Ti-Tai Wang (Microsoft)	+ cp312-abi3 (first abi3 wheel) · dropped cp39 + per-version cp312/cp313
1.21.0 Q1/26	27 March 2026	Sunny Anand (IBM)	+ cp314t (free-threaded 3.14, all platforms)
1.22.0 Q2/26	15 June 2026	Christian Bourjau (QuantCo)	+ Pyodide (cp312-abi3-pyemscripten) · dropped cp313t

IR Versions 11–13 & Type System Extensions

IR Versions

IR = the versioned spec defining ONNX's abstract model for graphs, operators, and their concrete format.

- **IR Version 11 – ONNX 1.18**
  - Added FLOAT4E2M1 (4-bit float for OCP MX microscaling)
  - Added multi-device proto classes for distributed inference
- **IR Version 12 – ONNX 1.19**
  - Added FLOAT8E8M0 (completes MX microscaling together with IR 11)
  - Multi-device proto specs formally documented in IR.md
  - FLOAT8E8M0 enabled for Q/DQ, CastLike, and other ops
- **IR Version 13 – ONNX 1.20**
  - Added INT2 and UINT2 — 2-bit integer types for ultra-low-bit quantized models
  - Enabled for Cast, CastLike, Q/DQ, and non-compute ops (Reshape, Transpose, Loop, Scan, etc.)

Type System

Defines the supported tensor element types — integers and floats at various precisions and bit-widths.

- **New types progressively enabled for Q/DQ, Cast, and non-compute ops**
  - Saturating cast semantics clarified for E4M3FNUZ / E5M2FNUZ
  - float16 / bfloat16 coverage extended across additional operators
- **BitCast**
  - Reinterpret tensor bit patterns without value conversion
- **Sub-byte tensor payload validation hardened**
- **Functions: graph-typed attributes added**
  - Enables higher-order functions and richer functional composition

Python/C++ Boundary: Stable ABI (abi3)

Impact: dramatically reduces per-release build matrix; downstream embedders get ABI stability

- **pybind11 replaced by nanobind — enables Python stable ABI wheels**
  - One compiled wheel binary works across Python 3.12, 3.13, 3.14, …
  - abi3audit integrated into release CI to verify compliance
  - Documented in README; downstream embedders benefit immediately

- **Explicit C++ symbol exports added**
  - ONNX_API annotations on protobuf symbols, schema registration functions
  - Functions consumed by PyTorch explicitly exported — resolves long-standing linkage issues
  - IsOnnxStaticRegistrationDisabled() fix for external runtime modules

- **cpp2py_export.cc refactored**
  - Binding boilerplate simplified; get_symbolic_input caster fixed
  - Cleaner boundary makes future pybind/nanobind evolution easier

Shape Inference · Spec Clarifications · New Targets

Shape Inference

Statically determines output tensor shapes from input shapes — enabling memory planning and graph optimisation without running the model.

- **unifyInputShape / unifyInputShapePrefix**
  – New InferenceContext methods — first-class API for output shape inference
  – Previously duplicated ad-hoc across operator implementations

- **Dim arithmetic: operator+ and operator- for Dim**
  – Algebraic dimension expressions in C++ inference code
  – Makes symbolic shape propagation across composite ops tractable

Spec & New Targets

- **Model domain: MUST → SHOULD**
  – Relaxes strict rejection of real-world models
- **Div integer: trunc semantics**
  – Aligns spec with actual runtime behaviour
- **DFT irfft, Shape bounds, NMS IoU, LpNorm 0/0 clarified**
- **.txtpb accepted as text proto extension**

- **Pyodide / WebAssembly wheels published**
  – ONNX usable in-browser and Pyodide runtimes
- **Python 3.13t / 3.14t free-threaded (all platforms)**
- **__repr__ for key proto classes · DataTypeSet → set[str]**

Infrastructure & Security Hardening — Highlights

Security

- **Supply-chain security**
  - OpenSSF Silver Badge achieved
  - SLSA Build Level 2 provenance attestations
  - zizmor lint for GitHub Actions — injection & over-privilege
  - PEP 770 build Software Bill of Materials in wheel — protobuf, nanobind, abseil-cpp versions + hashes
  - Reproducible builds documented and validated

- **Fuzzing infrastructure established**
  - Continuous fuzzing via OSS-Fuzz
  - Initial seed corpus for shape inference fuzzer

Build & Quality

- **Build system modernisation**
  - scikit-build-core, cibuildwheel, Pixi

- **C++ code quality**
  - clang-tidy CI job introduced; cpplint removed
  - OpenSSF compiler hardening flags
  - Microsoft Visual C++ warnings — first-class Windows build quality

- **Dependency automation**
  - Renovate for Pixi lockfile + Docker image updates


Pixi

SLSA

OpenSSF


Renovate

Forward Look — ROADMAP.md (Q3 2026 → Q2 2027)

Near term (Q3–Q4 2026)

- **Remove test files from release packages**
  – Reduces install surface and package size
- **Expand OSS-Fuzz Coverage Across More Targets**
- **Move to C++20**
  – Enables modern language features across the codebase
- **Immutable / tamper-evident releases**
  – Verifiable artifacts via cryptographic attestation

Horizon (Q1–Q2 2027)

- **Publish build SBOM alongside each release**
  – Complements the PEP 770 SBOM already included in each wheel
- **SLSA Build Level 3 compliance**
  – Hermetic, reproducible builds with verified toolchain
- **OpenSSF Gold Badge (Silver achieved)**
  – Comprehensive security posture across all criteria
- **Improved conformance test suite**
  – Parameterized, filterable, usable from non-Python runtimes
- **C++ hardening**
  – Compiler flags + static analysis integration

Key Takeaway

The SIG shifted from reactive maintenance to proactive supply-chain security and architectural modernisation.

Stable ABI + type system extensions + richer shape inference
make ONNX easier to embed, extend, and evolve
without breaking downstream consumers.

Get involved — ideas, needs, or just curious?
Reach out to me directly · Join our monthly SIG meeting

Thank you · Questions welcome

github.com/onnx/onnx · ROADMAP.md · RELEASE-MANAGEMENT.md